

Documentação do Sistema Melvin

Versão: 6fc154eacf91261386488e97ed27157a275e77e3

Data da Documentação: 22 de Maio de 2025

1. Visão Geral do Projeto

O "Sistema-Melvin" é um projeto desenvolvido para o Instituto Social Melvin, com o objetivo de realizar o cadastro e controle de frequência de alunos e voluntários.

2. Arquitetura do Sistema

O sistema é composto por três componentes principais: Frontend, Backend e Banco de Dados, orquestrados utilizando Docker.

2.1. Frontend

- **Tecnologia:** React com Vite.
- **Linguagem:** JavaScript (JSX).
- **Estilização:** SASS (SCSS) e CSS Modules (evidenciado pelos arquivos `*.module.scss`).
- **Principais Bibliotecas:**
 - `react`: Biblioteca principal para construção da interface de usuário.
 - `react-router-dom`: Para gerenciamento de rotas na aplicação.
 - `axios`: Para realizar requisições HTTP para o backend.
 - `js-cookie`: Para manipulação de cookies (provavelmente para autenticação).
 - `@mui/material`, `@emotion/react`, `@emotion/styled`: Componentes de UI baseados no Material Design.
 - `react-dropzone`: Para funcionalidade de upload de arquivos por "arrastar e soltar".
 - `react-icons`: Para utilização de ícones.
- **Build Tool:** Vite.

2.2. Backend

- **Tecnologia:** Spring Boot.
- **Linguagem:** Java (versão 21).
- **Principais Dependências (Frameworks/Bibliotecas):**
 - `spring-boot-starter-data-jpa`: Para persistência de dados utilizando JPA.
 - `spring-boot-starter-web`: Para desenvolvimento de aplicações web, incluindo APIs REST.

- **spring-boot-starter-security**: Para gerenciamento de autenticação e autorização.
- **postgresql**: Driver JDBC para conexão com o banco de dados PostgreSQL.
- **lombok**: Para reduzir código boilerplate (getters, setters, construtores, etc.).
- **java-jwt**: Para criação e validação de JSON Web Tokens (JWT).
- **spring-boot-devtools**: Para facilitar o desenvolvimento (ex: live reload).
- **ORM**: Hibernate (utilizado pelo Spring Data JPA).
- **Build Tool**: Maven.

2.3. Banco de Dados

- **Tipo**: PostgreSQL.
- **Nome do Banco**: **sistemamelvin**.
- As credenciais de acesso (**POSTGRES_USER**, **POSTGRES_PASSWORD**) estão definidas no arquivo **docker-compose.yml** (com valores ocultos).

3. Dockerização

O sistema é totalmente containerizado utilizando Docker e Docker Compose para facilitar a configuração e execução do ambiente de desenvolvimento e produção.

3.1. Serviços Definidos (**docker-compose.yml**)

- **postgresdb**:
 - **Imagem**: **postgresdb-melvin** (construída a partir de **./postgres**).
 - **Nome do Container**: **postgresdb**.
 - **Portas**: **5432:5432** (mapeia a porta 5432 do host para a porta 5432 do container).
 - **Volumes**: **postgres_data:/var/lib/postgresql/data** (persiste os dados do banco).
 - **Variáveis de Ambiente**: Define usuário, senha e nome do banco de dados.
 - **Limites de Recursos**: Memória limitada a 2G.
- **backend**:
 - **Imagem**: **backend-melvin** (construída a partir de **./sistema**).
 - **Portas**: **8080:8080**.
 - **Variáveis de Ambiente**: Configurações do Spring Boot para conexão com o banco de dados, JPA/Hibernate e dialeto SQL.
 - **Volumes**:
 - **./sistema:/app/src**: Mapeia o código fonte do backend para dentro do container, permitindo hot reload.

- `docs_diarios:/app/docs/diarios`: Volume para armazenamento de diários.
 - `docs_imagensembaixadores:/app/docs/imagens_embaxadores`: Volume para imagens de embaixadores.
 - `docs_imagensavisos:/app/docs/imagens_avisos`: Volume para imagens de avisos.
- **Depende de:** `postgresdb`.
- **Limites de Recursos:** Memória limitada a 1G.
- **frontend:**
 - **Imagem:** `frontend-melvin` (construída a partir de `./frontend`).
 - **Portas:** `80:5173` (mapeia a porta 80 do host para a porta 5173 do Vite dev server).
 - **Depende de:** `backend`.
 - **Volumes:**
 - `./frontend:/PORTFOLIO`: Mapeia o código fonte do frontend.
 - `/PORTFOLIO/node_modules`: Volume anônimo para `node_modules`, evitando que o `node_modules` local sobrescreva o do container.

3.2. Volumes Persistentes

- `postgres_data`: Armazena os dados do banco PostgreSQL.
- `docs_diarios`: Armazena arquivos de diários.
- `docs_imagensembaixadores`: Armazena imagens de embaixadores.
- `docs_imagensavisos`: Armazena imagens de avisos.

3.3. Script de Inicialização (`start.sh`)

O script `start.sh` automatiza o processo de build e inicialização dos containers Docker:

1. Para o serviço PostgreSQL (caso esteja rodando localmente).
2. Executa `docker-compose down` para parar e remover containers, redes, etc., da execução anterior.
3. Constrói as imagens Docker para `postgresqldb-melvin`, `backend-melvin` e `frontend-melvin`.
4. Executa `docker-compose up --build --force-recreate --remove-orphans` para subir os serviços, reconstruindo as imagens, forçando a recriação dos containers e removendo containers órfãos.

4. Configurações

4.1. Configurações do VS Code

- **launch.json:** Define a configuração de inicialização para debugar a aplicação Spring Boot (`SistemaApplication`) diretamente no VS Code.
 - `mainClass: br.com.melvin.sistema.SistemaApplication.`
 - `projectName: sistema.`
 - `envFile: ${workspaceFolder}/.env` (sugere o uso de um arquivo `.env` para variáveis de ambiente, embora este não tenha sido fornecido).
- **settings.json:**
 - `java.compile.nullAnalysis.mode: "automatic".`
 - `java.configuration.updateBuildConfiguration: "interactive".`

4.2. Configurações do Backend (`application.properties`)

- **Nome da Aplicação:** `spring.application.name=sistema.`
- **JPA/Hibernate:**
 - `spring.jpa.hibernate.ddl-auto: update` (o Hibernate tentará atualizar o schema do banco de dados com base nas entidades).
 - `spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect.`
 - `spring.jpa.properties.hibernate.show_sql = true` (exibe as queries SQL geradas no console).
 - `spring.jpa.properties.hibernate.format_sql = true` (formata as queries SQL exibidas).
- **DataSource (Conexão com Banco H2 - Local/Default, sobrescrito pelo `docker-compose.yml` em ambiente Docker):**
 - `spring.datasource.url=jdbc:postgresql://localhost/sistema melvin.`
 - `spring.datasource.username=****.`
 - `spring.datasource.password=*****.`
- **Segurança (JWT):**
 - `api.security.token.secret=${JWT_SECRET:*****}` (define o segredo para JWT, com um valor default).
- **Armazenamento de Arquivos:**
 - `file.upload-dir-diarios=docs/diarios/.`
 - `file.upload-dir-imagensembaixadores=docs/imagens_embaixadores/.`
 - `file.upload-dir-imagensavisos=docs/imagens_avisos/.`
- **URL do Frontend:**
 - `frontend.url=${FRONTEND_URL:http://192.168.18.20}` (URL do frontend, com valor default).

4.3. Classes de Configuração Java

- **FileStorageProperties.java**: Classe para carregar as propriedades de diretório de upload de arquivos (`file.upload-dir-*`) a partir do `application.properties`.
 - **SecurityProperties.java**: Carrega a propriedade `api.security.token.secret`.
 - **UrlFrontend.java**: Carrega a propriedade `frontend.url`.
 - **WebConfig.java**: Configura o Spring MVC para servir arquivos estáticos dos diretórios de documentos (`/app/docs/diarios/`, `/app/docs/imagens_embaixadores/`, `/app/docs/imagens_avisos/`).
-

5. Detalhes do Backend

5.1. Ponto de Entrada da Aplicação

- **SistemaApplication.java**: Classe principal que inicializa a aplicação Spring Boot.
 - A anotação `@SpringBootApplication` combina `@Configuration`, `@EnableAutoConfiguration` e `@ComponentScan`.
 - A anotação `@ConfigurationPropertiesScan("br.com.melvin.sistema.config")` habilita a varredura de classes anotadas com `@ConfigurationProperties` no pacote especificado.

5.2. Modelos de Dados (Entidades JPA)

Localizados em `br.com.melvin.sistema.model`:

- **AmigoMelvin.java**: Representa os "Amigos do Melvin".
 - Atributos: `id` (UUID), `nome` (String, não nulo), `contato` (String, não nulo), `email` (String), `formaPagamento` (String, não nulo), `valorMensal` (String, não nulo), `status` (Boolean, não nulo), `contatado` (Boolean, não nulo).
- **Aviso.java**: Representa os avisos do sistema.
 - Atributos: `id` (UUID), `titulo` (String, não nulo), `corpo` (String, não nulo), `status` (Boolean, não nulo), `data_inicio` (LocalDate, não nulo), `data_final` (LocalDate).
- **Cestas.java**: Modela as entregas de cestas básicas.
 - Atributos: `id` (UUID), `nome` (String, não nulo), `contato` (String), `rede` (String), `lider_celula` (String), `responsavel` (String), `dataEntrega` (LocalDate, não nulo).

- **Diario.java**: Representa os arquivos de diário de acompanhamento dos alunos.
 - Atributos: `id` (UUID), `matriculaAtrrelada` (String, não nulo, único), `fileName` (String, não nulo), `fileType` (String, não nulo), `filePath` (String, não nulo).
- **Embaixador.java**: Modela os embaixadores do instituto.
 - Atributos: `id` (UUID), `nome` (String, não nulo), `instagram` (String), `contato` (String, não nulo), `email` (String), `status` (Boolean, não nulo), `contatado` (Boolean, não nulo), `apelido` (String), `descricao` (String).
- **Imagem.java**: Entidade para armazenar metadados de imagens genéricas atreladas a outras entidades.
 - Atributos: `id` (UUID), `idAtrrelado` (UUID, não nulo, único - *Nota: o código indica `unique=true`, o que pode ser um problema se um mesmo `idAtrrelado` precisar de múltiplas imagens de tipos diferentes. A lógica no `ImagemService` de buscar por `idAtrrelado` E `tipo` sugere que essa unicidade pode não ser desejada ou é gerenciada de outra forma*), `fileName` (String, não nulo), `fileType` (String, não nulo), `filePath` (String, não nulo), `tipo` (String, não nulo - ex: "embaixador", "aviso").
- **frequencias/FrequenciaDiscente.java**: Registra a frequência dos alunos.
 - Atributos: `id` (UUID), `sala` (Integer, não nulo), `matricula` (String, não nulo), `nome` (String, não nulo), `presenca_manha` (String, não nulo), `presenca_tarde` (String, não nulo), `data` (LocalDate, não nulo), `justificativa` (String).
- **frequencias/FrequenciaVoluntario.java**: Registra a frequência dos voluntários.
 - Atributos: `id` (UUID), `matricula` (String, não nulo), `nome` (String, não nulo), `presenca_manha` (String, não nulo), `presenca_tarde` (String, não nulo), `data` (LocalDate, não nulo), `justificativa` (String).
- **integrantes/Discente.java**: Modela os alunos (discentes).
 - Contém diversos campos para informações pessoais, familiares, de saúde e institucionais, incluindo matrícula (única), nome, dados dos pais, informações de aulas extras (karate, ballet, etc.), e status.
- **integrantes/Voluntario.java**: Modela os voluntários.
 - Contém campos como matrícula (única), nome, contato, função, informações de disponibilidade semanal (segunda a sexta) e salas/aulas extras que leciona.

5.3. Repositórios (Spring Data JPA)

Localizados em `br.com.melvin.sistema.repository`:

- **AmigoMelvinRepository.java**: Interface para operações CRUD na entidade AmigoMelvin. Inclui método `findByNome`.
- **AvisoRepository.java**: Para Aviso. Inclui `findByTitulo`.
- **CestasRepository.java**: Para Cestas. Inclui `findByNomeAndDataEntrega` e `deleteByNomeAndDataEntrega`.
- **DiarioRepository.java**: Para Diario. Inclui `findByMatriculaAtrelada`.
- **EmbaixadorRepository.java**: Para Embaixador. Inclui `findByNome`.
- **ImagemRepository.java**: Para Imagem. Inclui `findByIdAtreladoAndTipo`.
- **frequencias/FrequenciaDiscenteRepository.java**: Para FrequenciaDiscente. Inclui `findByMatriculaAndData`, `deleteByMatriculaAndData`, `findAllByData`.
- **frequencias/FrequenciaVoluntarioRepository.java**: Para FrequenciaVoluntario. Inclui `findByMatriculaAndData`, `deleteByMatriculaAndData`, `findAllByData`.
- **integrantes/DiscenteRepository.java**: Para Discente. Inclui `findByMatricula`, `findByNome`, `deleteByMatricula`, `findAllBySala`, e uma query customizada `findTopOrderByMatriculaDesc`.
- **integrantes/VoluntarioRepository.java**: Para Voluntario. Inclui `findByMatricula`, `deleteByMatricula`, e uma query customizada `findTopOrderByMatriculaDesc`.

5.4. Camada de Serviço (Lógica de Negócio)

Localizados em `br.com.melvin.sistema.services`:

- **AmigoMelvinService.java**: Contém a lógica para listar, adicionar e alterar "Amigos do Melvin".
- **AvisoService.java**: Lógica para listar, adicionar e alterar avisos.
- **CestasService.java**: Gerencia o CRUD para entregas de cestas básicas.
- **DiarioService.java**: Lida com o upload, download, atualização e exclusão de arquivos de diário. Os arquivos são armazenados no diretório configurado em `file.upload-dir-diarios`.
- **EmbaixadorService.java**: Lógica para listar, adicionar e alterar embaixadores.
- **ImagemService.java**: Gerencia o upload e busca de imagens, diferenciando o diretório de upload com base no `tipo` ("embaixador" ou "aviso").
- **frequencias/FrequenciaDiscenteService.java**: Lógica para CRUD de frequências de alunos, com validações para campos obrigatórios e verificação de duplicidade e existência da matrícula.
- **frequencias/FrequenciaVoluntarioService.java**: Similar ao anterior, mas para frequências de voluntários.

- **integrantes/DiscenteService.java**: Lógica para CRUD de discentes, incluindo a geração automática de matrículas no formato ANOSEQUENCIAL (ex: 20240001).
- **integrantes/VoluntarioService.java**: Lógica para CRUD de voluntários, com geração de matrícula no formato ANO7SEQUENCIAL (ex: 202470001) e um método para listar voluntários com nome e função (DTO).

5.5. Controladores (APIs REST)

Localizados em `br.com.melvin.sistema.controller`:

- **AmigoMelvinController.java**: Endpoints para listar, adicionar e alterar "Amigos do Melvin" em `/amigomelvin`.
- **AvisoController.java**: Endpoints para CRUD de avisos em `/aviso`.
- **CestasController.java**: Endpoints para CRUD de cestas básicas em `/cestas`.
- **DiarioController.java**: Endpoints para upload, download, captura e exclusão de diários em `/diarios`.
- **EmbaixadorController.java**: Endpoints para listar, adicionar e alterar embaixadores em `/embaixador`.
- **ImagemController.java**: Endpoints para upload, atualização e listagem/captura de imagens em `/imagens`.
- **frequencias/FrequenciaDiscenteController.java**: Endpoints para CRUD de frequências de alunos em `/frequenciadiscente`, permitindo busca por data e/ou matrícula.
- **frequencias/FrequenciaVoluntarioController.java**: Similar ao anterior, mas para voluntários, em `/frequenciavoluntario`.
- **integrantes/DiscenteController.java**: Endpoints para CRUD de discentes em `/discente`, com listagem geral, por sala e captura por matrícula.
- **integrantes/VoluntarioController.java**: Endpoints para CRUD de voluntários em `/voluntario`, incluindo um endpoint para listar nomes e funções.

5.6. Segurança (Spring Security + JWT)

Localizados em `br.com.melvin.sistema.security`:

- **config/SecurityConfiguration.java**: Configuração principal do Spring Security.
 - Desabilita CSRF e configura a política de sessão para STATELESS (adequado para APIs REST com JWT).
 - Define as regras de autorização para os endpoints, especificando quais roles (COOR, PROF, AUX, COZI, ADM, MARK, ZELA, DIRE) têm acesso a quais recursos e métodos HTTP.
 - Configura o `SecurityFilter` para ser executado antes do `UsernamePasswordAuthenticationFilter`.
 - Define o `PasswordEncoder` como `Argon2PasswordEncoder`.

- Configura o CORS (Cross-Origin Resource Sharing) para permitir requisições de qualquer origem (*) e métodos HTTP comuns.
- **config/SecurityFilter.java**: Filtro que intercepta as requisições para validar o token JWT.
 - Recupera o token do header `Authorization`.
 - Valida o token usando o `TokenService` e, se válido, autentica o usuário no contexto de segurança.
 - Possui uma lista de endpoints públicos que não necessitam de autenticação.
- **config/TokenService.java**: Responsável por gerar e validar tokens JWT.
 - Utiliza o algoritmo HMAC256 com o segredo definido em `application.properties`.
 - Define um tempo de expiração para os tokens (5 horas).
- **controller/AuthenticationController.java**: Endpoints para autenticação e gerenciamento de usuários.
 - `/auth/login`: Autentica o usuário e retorna um token JWT.
 - `/auth/register`: Registra um novo usuário (voluntário) no sistema. Verifica se a matrícula do voluntário existe e se o login do usuário já não está registrado.
 - `/auth/alterar_senha/{matricula}/{senha}`: Altera a senha de um usuário.
 - `/auth/role_{matricula}`: Retorna o `UserRole` de um usuário pela matrícula.
 - `/auth/senha/{matricula}`: Retorna a senha (criptografada) de um usuário. *Esta prática não é recomendada por questões de segurança.*
 - `/auth/alterar_role/{matricula}/{role}`: Altera o `UserRole` de um usuário.
- **model/AuthenticationDTO.java**: DTO para dados de login (login e password).
- **model/LoginResponseDTO.java**: DTO para a resposta de login, contendo o token.
- **model/ResgisterDTO.java**: DTO para registro de novos usuários (login, password, role).
- **model/User.java**: Entidade JPA que representa um usuário do sistema para fins de autenticação, implementando `UserDetails`.
- **model/UserRole.java**: Enumeração para os diferentes papéis (roles) dos usuários no sistema (COOR, PROF, AUX, etc.).
- **repository/UserRepository.java**: Repositório para a entidade `User`. Inclui `findByLogin`.
- **services/AuthorizationService.java**: Implementação de `UserDetailsService` para carregar dados do usuário durante o processo de autenticação.

5.7. DTOs (Data Transfer Objects)

Localizados em `br.com.melvin.sistema.dto`:

- **VoluntarioDT0.java**: DTO simples para transferir nome e função de um voluntário.
-

6. Detalhes do Frontend

6.1. Estrutura e Configuração Inicial

- **frontend/index.html**: Ponto de entrada HTML da aplicação React.
 - Define o título da página como "Sistema Melvin".
 - Inclui a fonte "Roboto Flex" do Google Fonts.
 - Possui um `div` com `id="root"` onde a aplicação React será renderizada.
- **frontend/src/main.jsx**: Ponto de entrada JavaScript da aplicação. Renderiza o componente `App` no `div#root`.
- **frontend/src/App.jsx**: Componente raiz da aplicação que utiliza o `AppRoutes` para definir as rotas.
- **frontend/vite.config.js**: Arquivo de configuração do Vite.
 - Utiliza o plugin `@vitejs/plugin-react`.
 - Configura o servidor de desenvolvimento para escutar em `0.0.0.0`, permitindo acesso de outras máquinas na rede.

6.2. Gerenciamento de Rotas (**frontend/src/Routes.jsx**)

O arquivo `Routes.jsx` define a estrutura de navegação da aplicação utilizando `react-router-dom`. Ele separa as rotas em duas seções principais:

- **Rotas do Site Público (**SiteContent**)**: Acessíveis sem login, como `Home`, `MaisSobreNos`, `Embaixadores`, `Doacao`, etc. Estas rotas utilizam os componentes `HeaderSite` e `FooterSite`.
- **Rotas da Aplicação Interna (**AppContent**)**: Requerem autenticação e são protegidas por `PrivateRoute`. Incluem páginas de gerenciamento como `HomeApp` (dashboard específico por role), `Alunos`, `Voluntarios`, formulários de cadastro/edição, páginas de frequência e configurações. Estas rotas utilizam o componente `Header` da aplicação interna.

Principais Rotas Públicas:

- `/`: Página inicial do site.
- `/maissobrenos`: Página "Sobre Nós".
- `/embaixadores`: Página de Embaixadores.
- `/amigosmelvin`: Página "Amigos do Melvin".
- `/doacoes`: Página de Doações.

Principais Rotas da Aplicação (Internas - prefixo `/app/`):

- `/login`: Página de login.
- `/app/coord`, `/app/prof`, etc.: Dashboards específicos para cada perfil de usuário.
- `/app/alunos`: Listagem de alunos.
- `/app/voluntario/...`: Listagens e formulários de voluntários por tipo.
- `/app/embaixadores`: Listagem de embaixadores (para admin).
- `/app/amigosmelvin`: Listagem de amigos do Melvin (para admin).
- `/app/avisos`: Listagem e formulários de avisos.
- `/app/cestas`: Listagem e formulários de cestas básicas.
- `/app/frequencias/...`: Páginas para registro de frequência de alunos e voluntários.
- `/app/config`: Página de configurações do usuário e do sistema (para admin).
- `/app/rendimento_aluno/:matricula`: Página de rendimento do aluno.

6.3. Componentes Reutilizáveis

- **`frontend/src/components/Header/index.jsx`**: Cabeçalho da aplicação interna.
 - Exibe um ícone de menu (sanduíche) que aciona a `NavBar`.
 - Mostra o título "SISTEMA MELVIN" que redireciona para a página inicial do usuário logado.
 - Exibe o tipo de usuário logado (Coordenação, Professor, etc.) com base no cookie `role`.
- **`frontend/src/components/NavBar/index.jsx`**: Barra de navegação lateral.
 - Aparece quando o ícone de menu no `Header` é clicado.
 - Exibe links de navegação com base no perfil do usuário (`isAdm`, `isProf`, `isDire`, `isCoord`).
 - Utiliza o componente `SubNav` para submenus (ex: Voluntários).
 - Possui um ícone de configurações que redireciona para `/app/config`.
- **`frontend/src/components/SubNav/index.jsx`**: Componente para submenus dentro da `NavBar`.
 - Renderiza diferentes listas de links com base na prop `tipo` (ex: "voluntários").
- **`frontend/src/components/Deslogar/index.jsx`**: Botão para realizar logout.
 - Chama a função `auth.deslogar()` e redireciona para `/login`.
- **`frontend/src/components/gerais/Botao/index.jsx`**: Componente de botão genérico e customizável (nome, cor de fundo, cor da borda, comprimento, `onClick`, `tipo`).
- **`frontend/src/components/gerais/Input/index.jsx`**: Componente de input genérico e customizável (label, placeholder, tipo, nome, valor, `onChange`, comprimento, prioridade - para asterisco).
- **`frontend/src/site/components/HeaderSite/index.jsx`**: Cabeçalho para as páginas públicas do site.

- **frontend/src/site/components/FooterSite/index.jsx**: Rodapé para as páginas públicas do site. Inclui informações de contato e link para a área de login de voluntários.

6.4. Estrutura das Páginas

A pasta **frontend/src/pages** contém os componentes de página, divididos em:

- **Páginas de Usuário Logado (Dashboard e Funcionalidades):**
 - **Adm/, Aux/, Coor/, Cozi/, Dire/, Mark/, Prof/, Zela/**: Provavelmente representam as páginas iniciais para cada tipo de perfil de voluntário. Atualmente, a maioria parece ser placeholders.
 - **HomeApp/**: Dashboard principal da aplicação interna, exibindo quantidade de alunos presentes e avisos.
 - **Config/**: Página de configurações do usuário logado, permitindo visualização de dados pessoais e institucionais, registro de novos voluntários e alteração de senhas (para administradores), e acesso a listas de cadastros desativados. Inclui também uma seção de "Auto frequência".
 - **Rendimento/**: Página para visualização do rendimento de um aluno (provavelmente frequência e outras métricas).
 - **forms/**: Contém os formulários de cadastro e edição para **Aluno_forms**, **AmigoMelvin_forms**, **Aviso_forms**, **Cestas_forms**, **Embaixador_forms**, **Voluntario_forms**.
 - **frequencias/**: Contém as páginas para registro de **Aluno_frequencia** e **Voluntario_frequencia**.
 - **lista/**: Páginas para listagem de **Alunos**, **AmigosMelvinApp**, **Avisos**, **Cestas**, **EmbaixadoresApp**, **Voluntarios**.
 - **lista_matriculas_desativadas/**: Páginas para listagem de cadastros desativados (**AlunosDesativados**, **AmigosMelvinDesativados**, **AvisosDesativados**, **EmbaixadoresDesativados**, **VoluntariosDesativados**).
- **Páginas de Autenticação:**
 - **Login/**: Página para login dos usuários.
 - **Registro/**: Página para registrar novos usuários (voluntários) no sistema de autenticação. *Parece ser uma funcionalidade de desenvolvimento ou administrativa, já que o fluxo normal de cadastro de voluntário é feito por um formulário específico.*
- **Páginas do Site Público (frontend/src/site/pages/):**
 - **Home/**: Página inicial do site público.
 - **MaisSobreNos/**: Apresenta informações sobre o instituto, seus objetivos, missão, visão, valores e lista de voluntários.
 - **Embaixadores/**: Mostra os embaixadores ativos e como se tornar um.

- **AmigosMelvin/**: Explica o projeto "Amigos do Melvin" e direciona para o cadastro.
- **CadastroAmigo/**: Formulário para se tornar um "Amigo do Melvin".
- **SerEmbaixador/**: Formulário para interessados em ser embaixadores.
- **Doacao/**: Informações para doação (PIX e dados bancários).
- **NotaValor/**: Página sobre o programa "Sua nota tem valor".

6.5. Serviços do Frontend

Localizados em `frontend/src/services/`:

- **auth.js**:
 - **authentication(data)**: Envia credenciais para o endpoint `/auth/login` do backend. Armazena o token JWT e o login do usuário em cookies em caso de sucesso.
 - **registrar(dados)**: Envia dados para registrar um novo usuário no endpoint `/auth/register`.
 - **receberRole(matricula)**: Busca o papel (role) do usuário no endpoint `/auth/role_{matricula}`.
 - **deslogar()**: Remove os cookies de token e login.
- **http.js**: Configuração da instância do Axios.
 - Define a `baseUrl` a partir da variável de ambiente `VITE_REACT_APP_FETCH_URL`.
 - Configura um interceptor de requisições para adicionar automaticamente o token JWT (obtido do cookie) ao header `Authorization` de cada requisição, exceto para `/auth/login`.
- **PrivateRoute.jsx**: Componente de ordem superior para proteger rotas.
 - Verifica se o usuário está autenticado (token e login nos cookies) e se o `role` do usuário corresponde ao `role` exigido pela rota.
 - Redireciona para `/login` caso não autorizado.
- **verificacaoAuth.js**: Hook customizado para verificar a autenticação do usuário ao carregar a aplicação.
 - Se o usuário estiver autenticado, redireciona para a página correspondente ao seu `role`.
- **requests/ (subpasta)**: Contém módulos para requisições HTTP específicas:
 - **delete.js**: Funções para requisições DELETE (ex: `discente`, `voluntario`, `diario`, `frequenciadiscente`, `frequenciavoluntario`, `cestas`).
 - **get.js**: Funções para requisições GET (ex: `discente`, `voluntario`, `frequenciavoluntario`, `frequenciadiscente`, `diarioByMatricula`, `downloadFile`, `embaixadores`, `amigosmelvin`, `imagemPorId`, `imagemlista`, `aviso`, `cestas`).

- **post.js**: Funções para requisições POST (ex: `discente`, `voluntario`, `frequenciadiscente`, `frequenciavoluntario`, `uploadDiario`, `embaixadores`, `amigosmelvin`, `imagem`, `aviso`, `cestas`).
- **put.js**: Funções para requisições PUT (ex: `discente`, `voluntario`, `frequenciadiscente`, `frequenciavoluntario`, `atualizarDiario`, `alterarsenha`, `embaixadores`, `amigosmelvin`, `alterarRole`, `imagem`, `aviso`, `cestas`).

6.6. Estilização Global e Variáveis SASS

- **frontend/src/index.scss**: Arquivo SASS principal.
 - Define variáveis SASS globais para tamanhos de tela (breakpoints) como `$size-a`, `$size-b`, etc.
 - Define estilos base para `html`, `body`, `#root` e reseta margens e paddings.
 - Utiliza a fonte "Roboto Flex".

6.7. Scripts do `package.json`

- **dev: vite** (inicia o servidor de desenvolvimento do Vite).
- **build: vite build** (gera a build de produção).
- **lint: eslint . --ext js,jsx --report-unused-disable-directives --max-warnings 0** (executa o linter).
- **preview: vite preview** (pré-visualiza a build de produção localmente).
- **serve: serve -s dist -p 3000** (serve a build de produção na porta 3000, utilizando o pacote `serve`).

7. Execução e Deploy

1. **Construção das Imagens Docker:** O script `start.sh` primeiramente constrói as imagens Docker necessárias:
 - `docker build -t postgresql-melvin ./postgres`
 - `docker build -t backend-melvin ./sistema`
 - `docker build -t frontend-melvin ./frontend`
2. **Inicialização dos Serviços com Docker Compose:** O comando `docker-compose up --build --force-recreate --remove-orphans` é usado para iniciar todos os serviços definidos no `docker-compose.yml`.
 - O backend estará acessível em `http://localhost:8080`.
 - O frontend estará acessível em `http://localhost:80` (que internamente mapeia para a porta 5173 do Vite).
 - O banco de dados PostgreSQL estará acessível na porta `5432`.

3. Variáveis de Ambiente:

- O backend utiliza variáveis de ambiente para configurar a conexão com o banco de dados e o segredo JWT, que são fornecidas através do `docker-compose.yml`.
 - O frontend utiliza a variável de ambiente `VITE_REACT_APP_FETCH_URL` para definir a URL do backend.
-

8. Fluxos Principais e Funcionalidades

8.1. Autenticação

1. O usuário acessa a página de Login (`/login`).
2. Insere matrícula e senha.
3. Ao submeter, o frontend chama `auth.authentication()` que envia os dados para `/auth/login` no backend.
4. O backend (AuthenticationController) valida as credenciais contra a entidade `User`.
5. Se válido, o `TokenService` gera um token JWT.
6. O token é retornado ao frontend, que o armazena em cookies (`token` e `login`).
7. O frontend então busca o `role` do usuário (`auth.receberRole()`) e redireciona para o dashboard apropriado (ex: `/app/coor`, `/app/prof`).
8. Para requisições subsequentes a endpoints protegidos, o `http.js` intercepta e adiciona o token ao header `Authorization`.
9. No backend, o `SecurityFilter` valida este token.

8.2. Gerenciamento de Cadastros

- **Alunos:** Cadastro e edição através de `Aluno_forms`. Listagem em `Alunos` e `AlunosDesativados`. Os diários de acompanhamento podem ser upados e baixados.
- **Voluntários:** Cadastro e edição através de `Voluntario_forms`, segmentado por tipo de voluntário. Listagem em `Voluntarios` e `VoluntariosDesativados`.
- **Embaixadores:** Cadastro/edição em `Embaixador_forms` e listagem em `EmbaixadoresApp` e `EmbabaixadoresDesativados`. As imagens são gerenciadas.
- **Amigos do Melvin:** Cadastro/edição em `AmigoMelvin_forms` e listagem em `AmigosMelvinApp` e `AmigosMelvinDesativados`.
- **Avisos:** Cadastro/edição em `Aviso_forms` e listagem em `Avisos` e `AvisosDesativados`. As imagens são gerenciadas.
- **Cestas Básicas:** Cadastro/edição em `Cestas_forms` e listagem em `Cestas`.

8.3. Controle de Frequência

- Páginas dedicadas (`Aluno_frequencia`, `Voluntario_frequencia`) permitem o registro de presença (Manhã/Tarde - P, F, FJ) e justificativas para uma data específica.
- A página de `Config` permite que o voluntário logado registre sua própria frequência ("Auto frequência").

8.4. Configurações do Sistema

- A página `Config (/app/config)` permite:
 - Visualizar dados pessoais e institucionais do usuário logado.
 - Administradores (`ADM`) podem registrar novos voluntários no sistema de autenticação e alterar senhas de outros voluntários.
 - Acesso a listagens de alunos, voluntários, embaixadores e amigos do Melvin desativados.

8.5. Site Público

- Apresenta informações sobre o Instituto Melvin, formas de ajudar (Embaixador, Amigo do Melvin, Doações, Sua Nota Tem Valor) e páginas de cadastro para interessados.

9. Pontos de Atenção e Possíveis Melhorias

- **Segurança do Endpoint `/auth/senha/{matricula}`:** Expor a senha criptografada, mesmo que para administradores, não é uma prática recomendada. A alteração de senha deve ser feita sem a necessidade de conhecer a senha antiga, ou através de um fluxo de "esqueci minha senha" mais seguro.
- **Unicidade da `matriculaAtrrelada` em `Diario.java` vs. `idAtrrelado` em `Imagem.java`:** O campo `matriculaAtrrelada` em `Diario` é `unique=true`. O campo `idAtrrelado` em `Imagem` também é `unique=true`. Isso pode ser problemático se um mesmo `idAtrrelado` (por exemplo, de um embaixador) precisar de múltiplas imagens (talvez de tipos diferentes). A lógica de busca no `ImagemService` por `idAtrrelado` E `tipo` indica que a intenção pode ser permitir múltiplas imagens por `idAtrrelado` contanto que o `tipo` seja diferente, o que contradiz o `unique=true` no modelo `Imagem` para `idAtrrelado`.
- **Tratamento de Erros:** Embora haja `try-catch` em muitos lugares, a consistência e a forma como os erros são reportados ao frontend podem ser aprimoradas.
- **Validações:** As validações no backend (ex: `FrequenciaDiscenteService`) são importantes, mas podem ser complementadas com validações no frontend para melhor experiência do usuário.
- **Arquivo `.env`:** O arquivo `.vscode/launch.json` sugere o uso de um arquivo `.env`, mas este não foi fornecido. É importante para armazenar informações sensíveis localmente durante o desenvolvimento.

- **Hardcoding de URL no `http.js`:** A URL do backend (`import.meta.env.VITE_REACT_APP_FETCH_URL`) é configurada via variável de ambiente, o que é uma boa prática. O `application.properties` do backend também define `frontend.url`, mas o uso no CORS usa `addAllowedOriginPattern("*")` em vez de usar essa URL específica, o que pode ser um ponto de refinamento para produção.
- **Consistência de Nomenclatura:** Pequenas inconsistências na nomenclatura (ex: `EmbabaixadoresDesativados` em `Routes.jsx` vs. `EmbaixadoresDesativados` no nome do arquivo).

Esta documentação provê uma visão geral do Sistema Melvin com base nos arquivos fornecidos. Detalhes mais específicos de implementação de cada função podem ser encontrados nos respectivos arquivos de código-fonte.